

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project

COURSE NAME: ARTIFICIAL INTELLIGENCE EXPERT SYSTEMS COURSE AND CODE: MLA0101

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)
Assessment Tool Weightage: 40%

Total Marks: 100
No. of Questions: 1

Time: 90 Mins

Annexure A Expert System Development Project

Code of Conduct:

I Shaik Mubarak(192425194) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

Tool: SWI-Prolog

Usage of Prolog: Students shall use **Prolog** to develop a rule-based expert system by representing domain knowledge as **facts and production rules**, and implementing reasoning through **forward and backward chaining**. The project should demonstrate Prolog's **declarative/non-procedural nature**, logical inference, rule matching, and automated conclusion generation.

S.No.	Scenario-Based Expert System Development Question
1	Medical Diagnosis Expert System: Develop a rule-based expert system that identifies possible diseases based on symptoms, patient observations, and basic clinical information. Represent domain knowledge using production rules and implement both forward and backward chaining to derive conclusions.
2	Crop Disease Advisory System: Develop an expert system that identifies possible crop diseases based on leaf symptoms, soil conditions, weather conditions, and visible plant characteristics. Construct suitable production rules and demonstrate diagnosis using forward and backward chaining.
3	Automobile Fault Diagnosis System: Design an expert system that identifies probable vehicle faults based on symptoms such as engine noise, starting problems, warning indicators, abnormal vibration, and reduced mileage. Implement the knowledge base using production rules and compare forward and backward reasoning.
4	Student Academic Advisory System: Develop an expert system that recommends suitable academic interventions based on attendance, internal marks, assignment performance, learning difficulties, and previous academic performance. Use production rules and demonstrate both forward and backward chaining.
5	Cybersecurity Threat Diagnosis System: Construct a rule-based expert system that identifies possible cybersecurity threats from events such as repeated login failures, unusual network traffic, suspicious file activity, and unauthorized access attempts. Implement production rules and demonstrate how forward and backward chaining arrive at a threat classification.

Report Format:

Stage	Student Activity	Expected Output
1. Domain Analysis	Identify the problem domain, entities, facts, symptoms/conditions, and possible conclusions.	Problem definition and domain model
2. Knowledge Acquisition	Collect domain knowledge from reliable sources or domain experts.	Knowledge specification
3. Knowledge Representation	Convert domain knowledge into IF–THEN production rules.	Knowledge base
4. Inference Design	Implement forward chaining and backward chaining.	Inference engine
5. Paradigm Analysis	Distinguish how the solution can be expressed using procedural and non-procedural approaches.	Paradigm comparison
6. System Development	Integrate the knowledge base, inference engine, user interface, and explanation facility.	Working expert system
7. Testing	Test the system using multiple facts and scenarios.	Test cases and outputs
8. Evaluation	Analyze correctness, coverage, consistency, and reasoning performance.	Evaluation results
9. Documentation	Prepare technical documentation and GitHub repository.	Project report + GitHub
10. Demonstration	Demonstrate the system and explain the reasoning process.	Project presentation/viva

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, facts, conditions, goals, and expected conclusions.	7–8% – Good understanding of the domain with minor omissions.	5–6% – Basic domain understanding with several missing elements.	0–4% – Poor or incorrect understanding of the problem domain.
2. Knowledge Acquisition and Representation	10%	9–10% – Acquires relevant domain knowledge and represents it accurately using well-structured Prolog facts and rules.	7–8% – Knowledge is appropriately represented with minor gaps.	5–6% – Basic knowledge representation with missing or unclear facts/rules.	0–4% – Inadequate or inappropriate knowledge representation.
3. Production Rule / Knowledge Base Design	15%	13–15% – Develops a comprehensive, consistent, and logically structured production-rule knowledge base with appropriate facts and rules.	10–12% – Well-designed knowledge base with minor inconsistencies or omissions.	7–9% – Basic rule base developed but several rules are incomplete or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
4. Prolog Implementation	15%	13–15% – Implements a fully functional expert system using Prolog facts, predicates, rules, variables, unification, and backtracking	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

		effectively.			
5. Forward Chaining	10%	9–10% – Correctly implements and demonstrates forward chaining from initial facts through applicable rules to final conclusions.	7–8% – Forward chaining works correctly with minor limitations.	5–6% – Basic forward reasoning demonstrated but with incomplete rule processing.	0–4% – Forward chaining is missing or incorrectly implemented.
6. Backward Chaining	10%	9–10% – Effectively demonstrates goal-driven backward chaining using Prolog's inference mechanism and clearly traces the reasoning process.	7–8% – Backward chaining works correctly with minor gaps.	5–6% – Basic backward reasoning demonstrated with limited explanation.	0–4% – Backward chaining is missing or incorrectly implemented.
7. Procedural and Non-Procedural Paradigm Analysis	10%	9–10% – Clearly distinguishes procedural and declarative/non-procedural approaches and demonstrates their application in the Prolog system.	7–8% – Provides a good distinction with minor conceptual gaps.	5–6% – Basic distinction is explained but lacks practical demonstration.	0–4% – Paradigms are misunderstood or not addressed.

8. Inference, Unification and Reasoning Accuracy	10%	9–10% – Produces accurate conclusions using unification, backtracking, and logical inference across diverse test cases.	7–8% – Reasoning is mostly accurate with minor errors.	5–6% – Basic inference works but produces occasional incorrect results.	0–4% – Inference is unreliable or incorrect.
9. Testing, Results and System Demonstration	5%	5% – Provides comprehensive test cases, accurate outputs, reasoning traces, and a convincing live demonstration.	4% – Good testing and demonstration with minor gaps.	2–3% – Limited test cases and basic demonstration.	0–1% – Testing or demonstration is missing/inadequate.
10. Documentation, GitHub and Technical Presentation	5%	5% – Complete documentation, well-organized GitHub repository, clear code, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation/repository with several omissions.	0–1% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				

SCENARIO 3: AUTOMOBILE FAULT DIAGNOSIS SYSTEM

RULE-BASED AUTOMOBILE FAULT DIAGNOSIS EXPERT SYSTEM USING SWI-PROLOG

1. Domain Analysis

1.1 Problem Definition

Automobile vehicles can develop different faults such as battery failure, engine problems, overheating, brake problems, electrical faults, and abnormal vibration. Identifying the exact fault manually may require technical expertise and inspection. The proposed system is a rule-based expert system developed using SWI-Prolog that identifies probable automobile faults from observable symptoms and provides suitable recommendations. The system represents automobile knowledge using facts and IF–THEN production rules.

The system uses two reasoning approaches:

- **Forward chaining:** Starts with known symptoms and derives possible faults.
- **Backward chaining:** Starts with a suspected fault and checks whether the required symptoms are present.

1.2 Domain Entities

The major entities in the system are:

Entity	Examples
Vehicle	Car, Bike
Symptoms	Starting problem, engine noise
Observations	Warning light, vibration
Conditions	Weak battery, overheating
Faults	Battery fault, engine fault
Recommendations	Check battery, inspect engine

1.3 Input and Output

Input:

- Starting problems
- Engine noise
- Warning indicators
- Abnormal vibration
- Reduced mileage
- Overheating
- Brake problems
- Weak battery

Output:

- Probable automobile fault
- Supporting symptoms
- Recommended action

2. Knowledge Acquisition

Automobile troubleshooting knowledge is collected from basic vehicle maintenance principles and converted into logical rules suitable for a Prolog expert system.

The knowledge is divided into:

Symptoms

starting_problem
engine_noise
warning_light
abnormal_vibration
low_mileage
engine_overheating
brake_problem
weak_battery

Possible Faults

battery_fault
engine_fault
electrical_fault
brake_fault
cooling_system_fault
fuel_system_fault

Example Knowledge

Observed Conditions	Possible Fault
Starting problem + weak battery	Battery fault
Engine noise + low mileage	Engine fault
Warning light + weak battery	Electrical fault
Abnormal vibration + brake problem	Brake fault
Engine overheating	Cooling-system fault
Low mileage + engine noise	Fuel/engine fault

The acquired knowledge is represented as facts and production rules in Prolog.

3. Knowledge Representation

The system uses Prolog's declarative representation, where the programmer specifies what is true rather than describing every step of how to solve the problem.

3.1 Facts

Facts represent known observations about a vehicle.

starting_problem(car1).
engine_noise(car1).
weak_battery(car1).
warning_light(car1).
low_mileage(car1).

For example:

weak_battery(car1).
means that the battery of car1 is weak.

3.2 Production Rules

Production rules follow the general form:

IF condition(s) THEN conclusion

In Prolog:


```
fault(Vehicle, battery_fault) :-  
    starting_problem(Vehicle),  
    weak_battery(Vehicle).
```

This means:

IF a vehicle has a starting problem AND a weak battery, THEN the vehicle may have a battery fault.

3.3 Knowledge Base

A representative knowledge base contains rules such as:

```
fault(Vehicle, battery_fault) :-  
    starting_problem(Vehicle),  
    weak_battery(Vehicle).
```

```
fault(Vehicle, engine_fault) :-  
    engine_noise(Vehicle),  
    low_mileage(Vehicle).
```

```
fault(Vehicle, electrical_fault) :-  
    warning_light(Vehicle),  
    weak_battery(Vehicle).
```

```
fault(Vehicle, brake_fault) :-  
    abnormal_vibration(Vehicle),  
    brake_problem(Vehicle).
```

```
fault(Vehicle, cooling_system_fault) :-  
    engine_overheating(Vehicle).
```

This rule structure makes the knowledge base easy to modify and extend.

4. Inference Design

The inference engine is responsible for deriving conclusions from the available facts and rules. The system demonstrates both forward chaining and backward chaining.

4.1 Forward Chaining

Forward chaining is a data-driven reasoning method.

It starts with known facts and applies rules whose conditions are satisfied.

Example

Initial facts:

```
starting_problem(car1).  
weak_battery(car1).
```

Applicable rule:

```
fault(car1, battery_fault) :-  
    starting_problem(car1),  
    weak_battery(car1).
```

Reasoning:

Starting Problem
+
Weak Battery
↓
Production Rule

↓
Battery Fault

Thus, the system derives:

Battery Fault

Forward Chaining Algorithm

1. Read the initial facts.
2. Check the conditions of every rule.
3. Find rules whose conditions are satisfied.
4. Fire the applicable rules.
5. Generate new conclusions.
6. Continue until no new conclusion can be generated.

4.2 Backward Chaining

Backward chaining is a goal-driven reasoning method.

Instead of starting with all available facts, it starts with a goal and attempts to prove it.

For example, the goal is:

fault(car1, battery_fault).

Prolog searches for a rule that can establish this goal:

fault(Vehicle, battery_fault) :-
 starting_problem(Vehicle),
 weak_battery(Vehicle).

The system then checks the sub-goals:

Is starting_problem(car1) true?

↓
YES

Is weak_battery(car1) true?

↓
YES

Therefore:

battery_fault(car1)

Backward Chaining Query

?- fault(car1, battery_fault).

Output:

true.

This demonstrates Prolog's built-in backward-chaining inference mechanism.

5. Paradigm Analysis

5.1 Procedural Approach

In procedural programming, the programmer specifies how the problem should be solved using explicit control flow.

For example:

Read symptoms

- Check battery
- Check engine
- Check brakes
- Check warning light
- Display fault

The programmer controls the sequence of operations.

5.2 Non-Procedural/Declarative Approach

In Prolog, the programmer specifies facts and relationships/rules.

For example:

```
fault(Vehicle, battery_fault) :-
    starting_problem(Vehicle),
    weak_battery(Vehicle).
```

The programmer does not explicitly specify the search procedure. Prolog's inference engine performs unification, backtracking, and logical search automatically.

Comparison

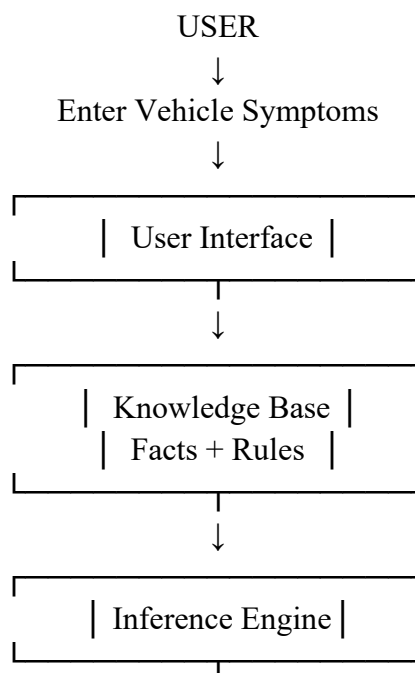
Procedural	Declarative Prolog
Describes how to solve	Describes what is true
Uses explicit control flow	Uses facts and rules
Programmer controls execution	Prolog controls inference
More algorithm-oriented	More knowledge-oriented
Usually uses loops/conditions	Uses logical inference
Less natural for rule systems	Well suited for expert systems

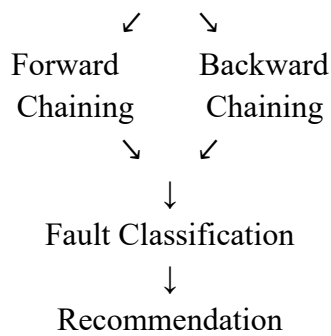
Therefore, Prolog is particularly suitable for developing this rule-based expert system.

6. System Development

The system is developed using **SWI-Prolog**.

6.1 System Architecture





```

SWI-Prolog console
[0] ?- [automobile_diagnosis].
Warning: C:/Users/shaik/Desktop/swi64/prolog/automobile_diagnosis.pl:27:
Warning: Classes of warning_light/1 are not together in the source file
Warning: Earlier definition at C:/Users/shaik/Desktop/swi64/prolog/automobile_diagnosis.pl:11
Warning: Current predicate: engine_overheating/1
Warning: Use :- discontiguous warning_light/1. to suppress this message
Warning: C:/Users/shaik/Desktop/swi64/prolog/automobile_diagnosis.pl:28:
Warning: Classes of abnormal_vibration/1 are not together in the source file
Warning: Earlier definition at C:/Users/shaik/Desktop/swi64/prolog/automobile_diagnosis.pl:22
Warning: Current predicate: warning_light/1
Warning: Use :- discontiguous abnormal_vibration/1. to suppress this message
Warning: C:/Users/shaik/Desktop/swi64/prolog/automobile_diagnosis.pl:29:
Warning: Classes of low_mileage/1 are not together in the source file
Warning: Earlier definition at C:/Users/shaik/Desktop/swi64/prolog/automobile_diagnosis.pl:17
Warning: Current predicate: abnormal_vibration/1
Warning: Use :- discontiguous low_mileage/1. to suppress this message
true.

[2] ?- show_symptoms(car1).
Symptoms for car1:
- Starting problem
- Weak battery
- Warning light
true.

[3] ?- backward_diagnosis(car1, battery_fault).
true.

[4] ?- fault(car1, battery_fault).
true.

[5] ?- forward_chain(car1).
.....
FORWARD CHAINING
Vehicle: car1
.....
Derived fault: battery_fault
Derived fault: electrical_fault
No known fault detected.
true.

[6] ?- diagnose(car1).
.....
AUTOMOBILE FAULT DIAGNOSIS SYSTEM
.....
Vehicle: car1

Possible fault: battery_fault
Recommendation: Check or replace the vehicle battery.

Possible fault: electrical_fault
Recommendation: Check battery connections, wiring and electrical system.

true.

[7] ?- diagnose(car2).
.....
AUTOMOBILE FAULT DIAGNOSIS SYSTEM
.....
Vehicle: car2

Possible fault: engine_fault
Recommendation: Inspect the engine and check engine components.

Possible fault: fuel_or_engine_fault
Recommendation: Inspect fuel system and engine performance.

true.

[8] ?- explain(car1, battery_fault).
Reason: Starting problem + weak battery
true.
  
```

6.2 Main Components

1. Knowledge Base

Contains automobile facts and production rules.

2. Inference Engine

Applies rules to derive conclusions.

3. User Interface

Allows the user to provide symptoms and request diagnosis.

4. Explanation Facility

Displays why a particular fault was identified.

5. Recommendation Module

Provides a basic suggested action based on the diagnosed fault.

7. Prolog Implementation

The system makes use of important Prolog concepts such as:

Facts

`engine_noise(car1).`

Rules

`fault(Vehicle, engine_fault) :-`

`engine_noise(Vehicle),`

`low_mileage(Vehicle).`

Variables

`Vehicle`

allows the same rule to work with different vehicles.

Unification

When the query:

`fault(car1, engine_fault).`

is evaluated, Prolog attempts to match it with:

`fault(Vehicle, engine_fault)`

and unifies:

`Vehicle = car1`

Backtracking

If one rule does not produce a solution, Prolog can automatically backtrack and search for another applicable rule.

This makes the system capable of exploring multiple possible diagnoses.

8. Testing

The system is tested using multiple scenarios to verify the correctness of the knowledge base and inference engine.

Test Case	Input Symptoms	Expected Diagnosis
TC1	Starting problem + weak battery	Battery fault
TC2	Engine noise + low mileage	Engine fault
TC3	Warning light + weak battery	Electrical fault
TC4	Abnormal vibration + brake problem	Brake fault
TC5	Engine overheating	Cooling-system fault
TC6	No matching symptoms	No known fault

Test Case 1

Facts:

starting_problem(car1).

weak_battery(car1).

Query:

?- fault(car1, battery_fault).

Output:

true.

Conclusion: The system correctly identifies a probable battery fault.

Test Case 2

Facts:

engine_noise(car2).

low_mileage(car2).

Query:

?- fault(car2, engine_fault).

Output:

true.

Conclusion: The system identifies a probable engine fault.

9. Evaluation

The system is evaluated based on correctness, coverage, consistency, and reasoning performance.

9.1 Correctness

The system produces a diagnosis when the required symptoms satisfy the corresponding production rule.

9.2 Knowledge Coverage

The knowledge base covers several common automobile problems including:

- Battery faults
- Engine faults
- Electrical faults
- Brake faults
- Cooling-system faults

Additional rules can easily be added.

9.3 Consistency

The rules are designed so that similar combinations of symptoms produce consistent conclusions.

9.4 Reasoning Accuracy

Prolog's unification and backtracking allow the system to correctly match variables with vehicle identifiers and explore alternative rules.

9.5 Forward vs Backward Reasoning

Feature	Forward Chaining	Backward Chaining
Direction	Facts → Conclusion	Goal → Facts
Approach	Data-driven	Goal-driven

Starts with	Known symptoms	Suspected fault
Useful for	Automatic diagnosis	Verifying a specific fault
Prolog support	Implemented using rule processing	Naturally supported by Prolog

10. Documentation and GitHub

The project documentation should contain:

1. Project title
2. Problem statement
3. Objectives
4. Domain analysis
5. Knowledge acquisition
6. Knowledge representation
7. Production rules
8. System architecture
9. Forward-chaining implementation
10. Backward-chaining implementation
11. Prolog concepts used
12. Test cases
13. Sample outputs
14. Evaluation
15. Limitations
16. Future enhancements
17. Conclusion
18. References

The GitHub repository can contain:

Automobile-Fault-Diagnosis/

```

|
|— automobile_diagnosis.pl
|— README.md
|— test_cases.txt
|— screenshots/
|   |— forward_chaining.png
|   |— backward_chaining.png
|— report.pdf

```

11. Limitations

The proposed system has some limitations:

- It uses a predefined knowledge base.
- It does not replace professional automobile inspection.
- Diagnosis depends on the accuracy of the entered symptoms.
- It covers only the faults represented by its rules.
- Multiple faults may require additional rules to distinguish between them.

12. Future Enhancements

The system can be extended by:

- Adding more automobile faults and symptoms.
- Supporting multiple vehicle types.
- Adding probability-based diagnosis.
- Developing a graphical user interface.
- Connecting the system to vehicle sensor data.
- Adding more detailed explanation facilities.
- Integrating real-time diagnostic information.

13. Conclusion

The Rule-Based Automobile Fault Diagnosis Expert System demonstrates how SWI-Prolog can be used to develop an intelligent system using facts, production rules, and logical inference. The system analyzes vehicle symptoms and derives probable faults using both forward and backward reasoning.

The project demonstrates important Prolog concepts including declarative programming, facts, rules, variables, unification, backtracking, and automated logical inference. Forward chaining provides data-driven diagnosis, while backward chaining allows the system to verify specific fault hypotheses.

Thus, the project demonstrates that Prolog's non-procedural and knowledge-based programming paradigm is well suited for developing rule-based expert systems.